

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №4

з дисципліни «Аналіз та рефакторинг коду»

на тему «Розробка IoT клієнта. Бізнес логіки та функцій налаштування»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н.І.

Прийняв:

асистент каф. ПІ

Дашенков Д.С.

Харків 2025

4.1 Мета роботи

Розробка та програмна реалізація клієнтської частини IoT-системи «Aquila». Метою є створення вбудованого програмного забезпечення для мікроконтролера ESP32, яке забезпечує зчитування фізичних показників (температура, вологість, рівень освітленості та шуму), детекцію руху, їх первинну обробку (перевірка на критичні значення, захист від спау/debounce), буферизацію даних при втраті зв'язку та передачу на віддалений сервер через API.

4.2 Засоби розробки та апаратне забезпечення

Для реалізації IoT-клієнта було обрано наступний стек технологій та обладнання.

Апаратна платформа (у симуляторі):

- мікроконтролер – ESP32 (Espressif Systems). Обраний завдяки наявності вбудованого Wi-Fi модулю для комунікації з сервером, високій продуктивності для обробки даних "на льоту" та достатньому обсягу пам'яті для реалізації буфера офлайн-даних.
- датчик середовища – DHT22. Цифровий датчик для вимірювання температури та вологості з високою точністю.
- датчик руху – PIR Motion Sensor. Використовується для виявлення несанкціонованого доступу в приміщення.
- датчик освітленості – LDR (Фоторезистор). Аналоговий датчик для вимірювання рівня світла в Люксах.
- датчик шуму (емуляція) – Потенціометр. Використовується для імітації рівня звукового тиску (dB) та тестування тривоги.

Програмне середовище:

- IDE – VS Code + розширення PlatformIO (прикладне середовище для розробки вбудованих систем);
- симулятор – Wokwi Simulator, що дозволяє емулювати роботу електронної схеми та мережеві запити без фізичного обладнання;
- мова програмування – C++ (фреймворк Arduino);
- бібліотеки – ArduinoJson (серіалізація даних у JSON), HTTPClient (відправка POST-запитів), DHTesp (робота з датчиком DHT).

4.3 Будова програмного забезпечення

Архітектура ПЗ клієнта побудована за модульним принципом:

- Hardware Abstraction Layer (HAL). Відповідає за ініціалізацію GPIO, зчитування цифрових сигналів з PIR-сенсора та аналогових значень з LDR/Потенціометра, а також взаємодію з бібліотекою DHTesp.
- Data Processing Module (Business Logic). Реалізує алгоритми перевірки порогових значень (TEMP_MAX, SOUND_MAX), конвертацію "сирих" значень АЦП у фізичні величини та логіку Debounce (кулдаун між сповіщеннями).
- Network & Buffer Manager. Відповідає за підключення до Wi-Fi, формування HTTP-запитів. Ключовою особливістю є механізм офлайн-буферизації: при відсутності мережі дані зберігаються у векторі (requestBuffer), а при відновленні з'єднання автоматично відправляються на сервер.

4.4 UML діаграма прецедентів

Для візуалізації функціональних можливостей розробленого програмного забезпечення та його взаємодії із зовнішнім середовищем (сервером) було побудовано діаграму прецедентів.



Рисунок 4.1 – Діаграма прецедентів

Діаграма відображає ключові сценарії роботи пристрою. Основним актором є мікроконтролер ESP32, який ініціює процеси збору даних. Система зчитує показники з сенсорів, виконує їх нормалізацію (Mapping) та перевірку на критичні значення (Edge Logic) перед відправкою. При спрацюванні датчика руху ініціюється сценарій «Детекція руху», який включає формування та відправку імітованого фотознімку (Snapshot). Реалізовано сценарій «Офлайн-буферизація» (через відношення <<extend>>). Якщо сервер недоступний, дані не втрачаються, а зберігаються локально. При наступному успішному підключенні спрацьовує сценарій «Синхронізація буферу».

4.5 Розробка бізнес-логіки

Реалізація математичної обробки даних на стороні клієнтського пристрою ESP32 є ключовим елементом для забезпечення достовірності інформації та зменшення навантаження на сервер.

Обробка показників освітленості (LDR): Аналоговий датчик освітленості повертає сире значення в діапазоні від 0 до 4095. Для представлення даних в універсальних одиницях Люкс (Lux) було застосовано функцію `map()`. Згідно з документацією та експериментальними даними, максимальне значення (4095) відповідає повній темряві, а мінімальне (0) – максимальному освітленню. Тому для коректного відображення рівня освітленості була використана інвертована масштабована залежність, яка відображає світловий потік у діапазоні від 0 до 10000 Люкс.

Обробка рівня шуму (Кнопка-симулятор): Аналоговий вхід, до якого підключено потенціометр, використовується для емуляції рівня звукового тиску. Стан цифрової кнопки, що перевіряється (`digitalRead(PIN_BTN_NOISE)`), інтерпретується як сигнал про рівень шуму. Коли кнопка натиснута (стан LOW через підтягуючий резистор), це відповідає високому рівню шуму (симуляція 80-100 dB). Якщо кнопка не натиснута (стан HIGH), пристрій генерує значення фонового шуму (30-50 dB). Це дозволяє тестувати механізм тривоги при перевищенні порогових значень гучності.

Фільтрація подій детекції руху: Для запобігання надмірній кількості запитів на сервер при виявленні руху реалізовано механізм "охолодження". Після першого спрацювання датчика руху та відправки відповідного запиту запам'ятовується час події. Наступні виявлення руху ігноруються протягом певного періоду. Це дозволяє відправляти лише первинний сигнал про вторгнення, не перевантажуючи систему зайвими даними.

4.6 Діаграма діяльності

Діаграма демонструє алгоритм роботи головного циклу (loop) мікроконтролера. Алгоритм побудований на неблокуючих таймерах, що дозволяє паралельно виконувати три незалежні процеси: синхронізацію буфера накопичених даних, обробку подій безпеки (рух) та періодичний збір телеметрії.

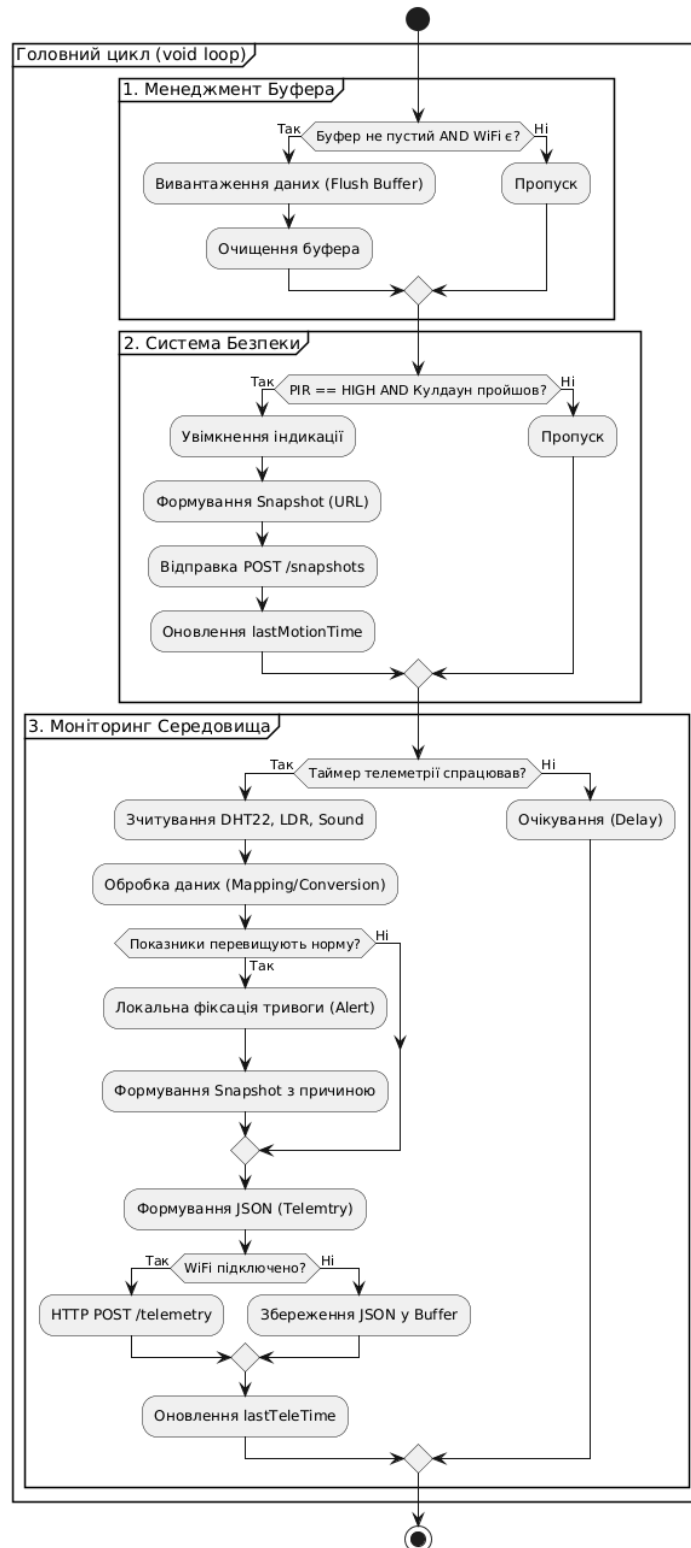


Рисунок 4.2 – Діаграма діяльності

4.7 Програмна реалізація

Програмна реалізація виконана мовою C++ з використанням фреймворку Arduino. Для забезпечення надійності передачі даних в умовах нестабільного зв'язку було розроблено механізм офлайн-буферизації.

```
struct BufferedRequest {
    String endpoint;
    String json;
};

std::vector<BufferedRequest> requestBuffer;

void sendRequest(String endpoint, DynamicJsonDocument& doc) {
    String json;
    serializeJson(doc, json);

    if (!connectWiFi()) {
        if (requestBuffer.size() < MAX_BUFFER_SIZE) {
            BufferedRequest req;
            req.endpoint = endpoint;
            req.json = json;
            requestBuffer.push_back(req);
            Serial.print(" Buffered request to: ");
            Serial.println(endpoint);
        } else {
            Serial.println(" Buffer full! Data lost.");
        }
        return;
    }
    // ... далі йде стандартна відправка HTTP
}
```

Для відновлення синхронізації з сервером реалізовано функцію, яка автоматично вивантажує накопичені дані при появі мережі.

```
void flushBuffer() {
    if (requestBuffer.empty() || WiFi.status() != WL_CONNECTED)
        return;

    Serial.println(" Flushing buffer...");

    while (!requestBuffer.empty()) {
        BufferedRequest req = requestBuffer.front();

        HTTPClient http;
        http.begin(SERVER_URL + req.endpoint);
```

```

http.addHeader("Content-Type", "application/json");

int code = http.POST(req.json);

if (code > 0) {
    Serial.print(" Synced: "); Serial.println(req.endpoint);
    requestBuffer.erase(requestBuffer.begin()); // Видаляємо
    успішно відправлений елемент
} else {
    break; // Якщо знову помилка - перериваємо вивантаження
}
http.end();
}
}

```

Клієнтська частина реалізує власну бізнес-логіку (Edge Computing), перевіряючи показники на критичні значення безпосередньо на пристрої. Це дозволяє миттєво реагувати на небезпеку (наприклад, критичне підвищення температури) ще до обробки даних сервером.

```

if (now - lastAlertTime > COOLDOWN) {

    if (dht.getStatus() == 0) {
        if (temp < TEMP_MIN) {
            sendSnapshot("Low Temperature (" + String(temp, 1) + "°C");
            hasAlert = true;
        } else if (temp > TEMP_MAX) {
            sendSnapshot("High Temperature (" + String(temp, 1) +
"°C");
            hasAlert = true;
        }
    }

    if (sound > SOUND_MAX) {
        sendSnapshot("High Noise (" + String(sound) + " dB");
        hasAlert = true;
    }
}
}

```

Для перевірки працездатності розробленого програмного забезпечення було проведено серію тестів у середовищі симуляції Wokwi. На першому етапі було перевірено коректність підключення компонентів згідно з розробленою схемою. На рисунку нижче зображено загальний вигляд пристрою в симуляторі, що включає мікроконтролер ESP32, датчик

температури та вологості DHT22, датчик руху PIR, фоторезистор та кнопку для емуляції рівня шуму.

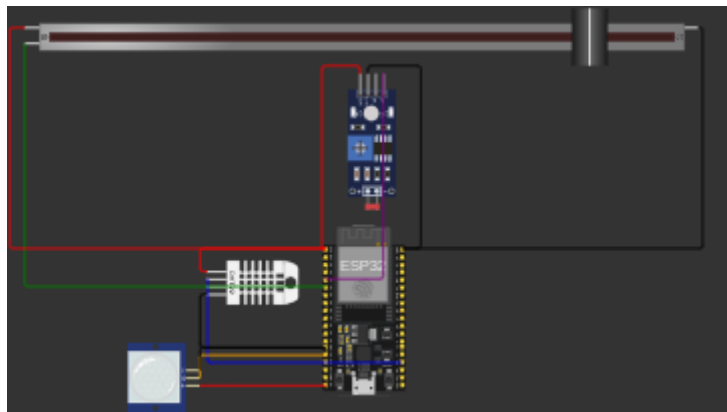


Рисунок 4.3 – Схема підключення компонентів IoT-клієнта

Далі було протестовано процес ініціалізації пристрою. У моніторі послідовного порту (Serial Monitor) зафіксовано успішну перевірку статусу сенсорів («GPIO OK», «DHT22») та підключення до бездротової мережі з отриманням IP-адреси.

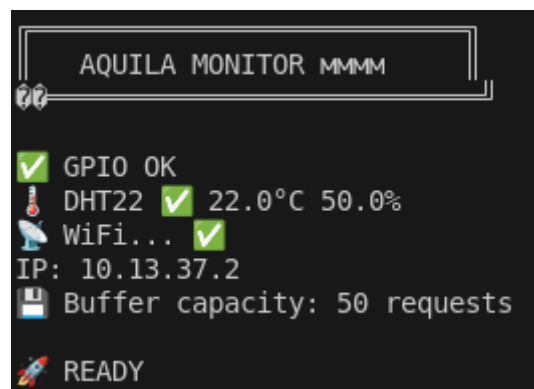


Рисунок 4.4 – Логи ініціалізації системи

Перевірка основного циклу роботи продемонструвала коректність збору та відправки телеметрії. Пристрій кожні 10 секунд зчитує показники, формує JSON-документ та отримує код відповіді 200 або 201 від сервера, що свідчить про успішне збереження даних у базі. Також протестовано роботу системи безпеки: при активації датчика руху (PIR) або натисканні кнопки емуляції шуму пристрій миттєво генерує тривожне повідомлення (Snapshot) з описом причини («Unauthorized Motion» або «High Noise»).

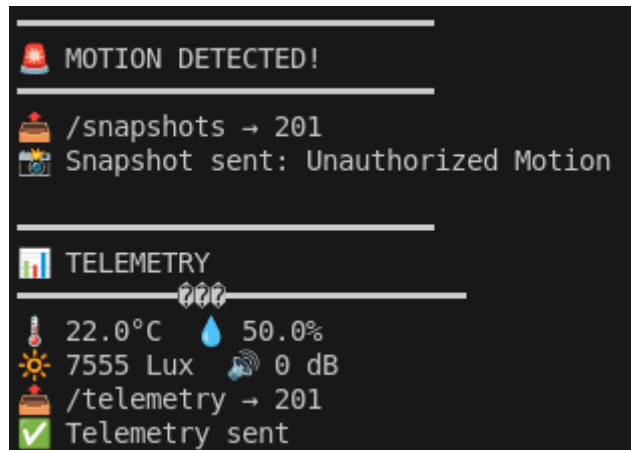


Рисунок 4.5 – Логування відправки телеметрії та реакції на рух

4.9 Висновки

У ході виконання лабораторної роботи було успішно спроектовано та реалізовано клієнтську частину IoT-системи «Aquila» на базі мікроконтролера ESP32. Створено прошивку мовою C++ у середовищі PlatformIO, яка забезпечує взаємодію з комплексом сенсорів (DHT22, PIR, LDR) та мережевий обмін даними. Впроваджено локальну бізнес-логіку на стороні клієнта. Пристрій самостійно виконує первинну обробку сигналів, конвертацію фізичних величин та аналіз критичних показників, що зменшує навантаження на центральний сервер. Ключовим досягненням роботи є реалізація механізму офлайн-буферизації. Використання динамічних структур даних (std::vector) дозволило забезпечити накопичення даних при втраті зв'язку та їх автоматичну синхронізацію при відновленні мережі, що є критично важливим для систем безпеки. IoT-клієнт успішно інтегровано з раніше розробленим бекендом. Дані коректно передаються через REST API, а система автоматично реагує на події руху та перевищення порогових значень шуму чи температури. Розроблений клієнт є повністю функціональним, автономним та готовим до розгортання у складі комплексної системи моніторингу.